

MBAX 6418 — Assignment 1: Sentiment & Emotion Classification of Amazon Reviews

Goal: Produce a working review-sentiment classifier for Amazon reviews — it classifies each review as positive / neutral / negative, detects the primary emotion, checks its predictions against the star rating, and presents everything in a polished dashboard.

How to work on this: drive it with an **agent** — each step is a goal to reach, not a checklist. You and the agent choose the approach as you go: API, tooling, structure, dashboard design. A few fixed choices: the classification and scoring code is written in **Python**, and model calls go through an **OpenAI-compatible** endpoint (e.g., the one we used to set up Hermes Agent). The dashboard itself can be HTML or anything else. Work **in order** — each step should leave you with something working. Your **classmates** are fair game for comparing approaches, asking questions, and teaming up — but the write-up is owned by whoever submits it, and everyone submits their own copy (see the GitHub section for how that works).

A note: where an earlier step hints at a later "known finding," treat it as a hunch to test — it's more interesting if your numbers disagree.

(Useful if you haven't seen them: OpenAI offers verified US/Canada students \$100 in ChatGPT credits for use in Codex — <https://developers.openai.com/community/students>.)

The data

The task uses the **Amazon 2023 "Gift Cards" review category** — part of the large-scale **Amazon Reviews '23** dataset collected by the [McAuley Lab](#) at UC San Diego. You can learn more about the dataset (its scale, other categories, and the full field reference) on the dataset's site:

```
https://amazon-reviews-2023.github.io
```

The Gift Cards review file itself lives on the McAuley Lab's public dataset host:

```
https://mcauleylab.ucsd.edu/public_datasets/data/amazon_2023/raw/review_categories/Gift_Cards.jsonl.gz
```

It is a gzipped **JSON Lines** (newline-delimited JSON) file; each line is one review. Expected fields:

- `rating` — the star rating, a number in `{1, 2, 3, 4, 5}` (encoded as a float, e.g., `4.0` ; integral values throughout this dataset).
- `title` — the short review-issue title.
- `text` — the review body.
- `verified_purchase` — boolean, whether the reviewer actually bought the item.
- `helpful_vote` — integer count of "helpful" votes for the review.
- `timestamp` — Unix milliseconds when the review was written.
- `images` — array; image references attached to the review (often empty).
- `asin` — the product this review is about.
- `parent_asin` — the product family / parent item.
- `user_id` — the reviewer.

Use your judgment about metadata worth keeping. Before building anything, confirm you can actually read the file.

Step 1 — A structured prompt

A reusable prompt that takes a review's **title** and **text** and asks an LLM to classify it as **POSITIVE** or **NEGATIVE**. Decide the edge cases yourself (conflicting title/text, terse or angry short reviews); the prompt must be set up so the model gives you a clear, simple answer you can read back programmatically, and it must not rely on the rating.

Ask yourself: does a quick spot-check on obviously positive and negative reviews come out right?

Step 2 — Score against the rating

Score a **100-row** first batch and see how the model does against the "correct answer" you work out **from the rating** (≥ 4 positive, else negative). The model must **never see the rating** — it's only for checking afterwards. How you structure and save the work is up to you; at minimum, be able to say how often the model agrees with the rating, which reviews it gets wrong, and a feel for how often it's right on each class.

Ask yourself: can you say, with evidence, what the model is good and bad at? Keep in mind the data is heavily skewed toward high ratings — most reviews are ★★★★★ or ★★★★★★ — and don't let that lopsidedness fool you into thinking the model is better than it is.

Step 3 — A results dashboard

Present the results in a visual dashboard. A **single self-contained HTML page** is a good option — everything baked into one file, works offline, no server or network needed — but you're not limited to it: interactive app frameworks like **Streamlit** and **Gradio** work just as well. Decide what belongs in it; at minimum the headline numbers, a clear picture of which answers were right and wrong and how mistakes cluster by class, and enough per-review detail to back up what you claim.

Design bar: this is a product, not a demo. Deliberate palette, refined typography, a coherent theme the host can recolor — no library-shuffle look.

Ask yourself: do the numbers on the page match what your scoring produced, and can a non-technical reader tell at a glance how good the model is?

Step 4 — Interactive review filtering

Make the review table explorable: let a reader filter to a subset (e.g., correct vs mismatched) with a live count. Whether you do more than that is up to you.

Ask yourself: does filtering show exactly the rows you asked for, and does the count make sense with what you recorded?

Step 5 — Primary-emotion detection

Add **two independent** takes on the review's primary emotion:

1. **The LLM predicts it** — extend the prompt's output so each review gives both sentiment and a primary emotion.
2. **A word list derives it** — score each review's words against an **NRC emotion word list** (a public list linking words to emotions: anger, anticipation, disgust, fear, joy, sadness, surprise, trust), add the scores per emotion, and take the highest as the answer. This needs no model calls and runs over your existing predictions.

Keep both, and compare them.

Ask yourself: how often do the two agree, and where/why do they diverge?

Step 6 — Three-class scoring with balanced sampling

Redefine to the full split:

Rating	Class
4–5	POSITIVE
3	NEUTRAL
1–2	NEGATIVE

Carry the change through everything — the correct answer you work out, the numbers you track, the prompt, and how the dashboard shows it. Because reading the first N rows in order under-represents the rarer classes, instead pull a **balanced group** from the whole file — a roughly equal number from each class, picked with a **fixed random seed** so the same set comes up every time — around 50 per class.

Ask yourself: what do the balanced three-class numbers actually show, and what did the balanced run reveal that the imbalanced one hid? In particular, do ★★★ (3-star) reviews get their own class, or do they collapse into another?

Step 7 — Descriptive and prediction visualizations

Give the dashboard a descriptive layer — e.g., a star-rating distribution, a comparison of the "correct answer" versus what the model predicted for each class, and how often each class was answered right. The point is to make the model's failures visible at a glance.

Two things to be careful about:

- Check the numbers on the page against the numbers you saved earlier — in the browser, not just by eye.
- Watch out for layout bugs that can collapse very small chart elements (e.g., bars given zero width by a label/positioning interaction).

Ask yourself: can a reader see, without drilling in, where the model succeeds and fails?

Final deliverables

Produce a **short report** plus the working files: the prompt, the scoring script that runs the reviews, the word-list script that adds the emotions, the dashboard generator, one balanced

run's raw output, and the final dashboard.

Report format: write it as **markdown** in a `README.md` (so it can later drop into a GitHub repository as-is), and **include one or more screenshots** of at least the interface (for example, a capture of the dashboard you built). Make sure every number you quote in it matches a visible figure in the saved output. **Also cite the source of the data** (the Amazon Reviews '23 dataset and its page) inside the README — don't expect a reader to go hunting for where the reviews came from.

The report should be able to answer, with evidence:

1. Why did the lopsided run look very accurate, and what did sampling equal amounts of each class change?
2. Look at where the model's mistakes go — which classes get confused with which, and in what direction? (For example, do ★★★ (i.e., 3-star neutral) reviews get labeled negative, or do negative ones get called neutral?) Report the concrete numbers from your matrix.
3. How do the LLM's emotions and the word list's emotions differ, and why?
4. What bugs and/or issues did you hit along the way — in the charts, the UI, in the process of working with the agent, or anywhere else — and how did you work around them?

Author / review split: the report should be **generated with the agent** (it produces the draft, pulls the numbers from the saved output, writes the narrative), but it is **your** deliverable — **review and check it yourself** before submitting. Checking the claimed numbers against the saved output and putting the agent's framing and conclusions into your own words is part of the assignment, not an optional pass.

Submission: publish to GitHub and share the link

Requirement: the final `README.md` report **and all the project code** must live in a **GitHub repository**, and you must hand in a **shareable link** to it. There is no alternative submission path — the repo *is* the submission.

One per person, even in a team. If you work with classmates on the same work, you don't all share one repo and one hand-in — **each of you submits your own repository**, even if it contains essentially the same project. That requirement is there on purpose: it verifies that GitHub access works for *every* person in the class, not just one or two per group. If a teammate pushes and everyone else just forwards the same link, only that teammate's account has proven GitHub works.

Public or private? Public is the simplest default — anyone with the link can read it, no extra steps. Private works too, but it costs you one small step: add me

(`david.dobolyi@colorado.edu`) as a collaborator on the repo, otherwise I can't see the files when I open it. Do that after you create the repo, before you hand in the link.

Where to submit the link. The repo link is handed in as the assignment submission on **Canvas** — put the shareable GitHub link in the Canvas submission so it's tied to your account there. Just make sure the link you paste works (see the check further down) before you hit submit.

New to GitHub? That's expected and fine — you'll likely need to create an account. Sign up here if you don't have one:

```
https://github.com/signup
```

Because you're all students, the **GitHub Student Developer Pack** gives you free access to tools that normally cost money — including GitHub **Pro**, **Copilot**, and **Codespaces** credits — plus bundled tutorials for getting started with git and open source:

```
https://education.github.com/pack
```

The agent can and should help with every step. It can't sign up for you (only you can), but it can:

- guide you through account creation and the one-time setup (creating a token / authenticating `git`),
- initialize a repository in your project folder,
- stage and commit your code and the report,
- create the GitHub repository and push to it,
- for a private repo, invite me (`david.dobolyi@colorado.edu`) as a collaborator,
- tell you how to confirm it's accessible and grab the shareable link.

Before you push, be deliberate about what goes in: include the code and the `README.md` ; think about whether the data file (which is large and re-downloadable) or any credentials/tokens belong in the repo — generally they don't. Ask the agent if you're unsure what's safe to commit.

Ask yourself: does the link you submit open a repository that *you* can access, and would the person grading it be able to too — a public repo readably, or a private one after you've invited them? Can they see the README (screenshots and all), and is the code there enough to reproduce your results?

Standing considerations

- **The rating is the correct answer:** it's what you judge the model against, but the model never sees it.
- **Make your results repeatable:** any number you publish should come from a fixed choice of which reviews were used (a fixed "seed") and fixed settings, so running it again gives the same answer.
- **Show your work:** every number you claim should match the saved output — don't publish something you haven't re-checked.
- **Be honest about imbalance:** say openly how unbalanced the data is; never let a run that's mostly one easy class look like proof the model is great.
- **Product bar:** the dashboard is a finished, cohesive product.